

智慧银行实验教程

实验三：OpenBB 金融分析、BMAD 技能开发与银行系统构建

何彦霖 金融 202301 202301746

2026 年 7 月 1 日

1 实验目的

本次实验旨在深入学习和掌握金融数据分析、AI 技能开发和 Web 应用构建等多项核心技术。具体学习目标包括：

- 掌握 OpenBB Finance 技能的安装与使用，进行投资价值分析
- 理解 BMAD 方法学和技能包的安装与集成
- 开发带难度等级、界面选择和蛇风格的贪吃蛇网页游戏
- 分析贪吃蛇网页游戏的市场前景和市场化可能性
- 掌握 Vercel 生产环境部署流程
- 安装 MiKTeX 和 LaTeX 插件，实现 PDF 文档编译
- 使用 BMAD 技能生成银行 CRM 系统

2 实验环境

项目	配置
操作系统	Windows 10/11
Node.js	v20+
npm	10.2.4+
Python	3.9-3.12
开发工具	Trae IDE / Visual Studio Code
前端技术	HTML5, CSS3, JavaScript (ES6+)
构建工具	Vite 5.0+
部署平台	Vercel
LaTeX 编译器	MiKTeX (xelatex)
版本控制	Git + CNB CLI
创建时间	2026-06-18

表 1: 实验环境配置

3 实验步骤

3.1 步骤 1: OpenBB Finance 技能安装与 SpaceX 投资价值分析

3.1.1 技能安装

OpenBB 是一个开源的金融数据平台，提供股票、期权、加密货币、宏观经济指标等多维度金融数据。安装命令如下：

```
1 pip install openbb
2 pip install openbb-cli
```

3.1.2 技能配置文件

技能文件位于 c:\Users\25590\Documents\123\smartbanking\.agents\skills\openbb-finance 核心功能包括：

功能模块	描述
股票数据获取	历史股价、实时行情、技术指标
期权数据	期权链、隐含波动率、希腊字母
加密货币	虚拟货币价格、交易量
宏观经济	GDP、CPI、失业率等经济指标
基本面分析	财务报表、估值指标
AI 集成模式	结构化数据返回，支持 AI 代理调用

表 2: OpenBB Finance 技能功能

3.1.3 SpaceX 投资价值分析代码

```

1 from openbb import obb
2 import pandas as pd
3 import pandas_ta as ta
4
5 class StockAnalyzer:
6     def __init__(self):
7         self.obb = obb
8
9     def analyze_stock(self, symbol, period="1y"):
10        output = self.obb.equity.price.historical(
11            symbol=symbol,
12            period=period
13        )
14        df = output.to_dataframe()
15
16        current_price = df['close'].iloc[-1]
17        total_return = (df['close'].iloc[-1] / df['close'].iloc[0] -
18            1) * 100
19
20        df['sma_20'] = ta.sma(df['close'], length=20)
21        df['sma_50'] = ta.sma(df['close'], length=50)
22        df['rsi'] = ta.rsi(df['close'], length=14)
23
24        volatility = df['close'].pct_change().std() * (252**0.5)
25
26        return {
            "symbol": symbol,

```

```
27         "current_price": round(current_price, 2),
28         "period_return_percent": round(total_return, 2),
29         "volatility": round(volatility, 4),
30         "rsi": round(df['rsi'].iloc[-1], 2),
31         "data_points": len(df)
32     }
33
34 # 分析SpaceX相关股票（如特斯拉作为参考）
35 analyzer = StockAnalyzer()
36 result = analyzer.analyze_stock("TSLA")
37
38 print(f"分析结果: {result}")
```

3.1.4 投资价值分析要点

- 1. 公司概况：SpaceX 作为马斯克旗下的航天科技公司，致力于太空探索和卫星互联网
- 2. 市场定位：在商业航天领域具有领先地位，Starlink 项目具有巨大市场潜力
- 3. 财务指标：需关注营收增长、盈利能力、研发投入等关键指标
- 4. 风险评估：技术风险、政策风险、市场竞争风险等
- 5. 投资建议：基于技术分析和基本面分析给出买入/持有/卖出建议

3.2 步骤 2：BMAD 技能包安装

3.2.1 BMAD 方法学简介

BMAD（Build-Measure-Adapt-Deliver）是一套系统化的产品开发方法学，包含以下核心模块：

模块	技能
分析阶段	bmad-agent-analyst, bmad-document-project, bmad-market-research
规划阶段	bmad-agent-pm, bmad-create-prd, bmad-ux
方案阶段	bmad-agent-architect, bmad-create-architecture
实施阶段	bmad-agent-dev, bmad-quick-dev, bmad-code-review
核心技能	bmad-brainstorming, bmad-spec, bmad-customize

表 3: BMAD 技能模块

3.2.2 安装命令

从 GitHub 克隆 BMAD 项目并安装为项目级技能包：

```
1 git clone https://github.com/.../bmad.git
2 cd bmad
3 npm install
4 # 通过BMAD CLI安装技能
5 node tools/installer/bmad-cli.js install
```

3.2.3 技能包结构

```
BMAD/
  src/
    bmm-skills/
      1-analysis/      # 分析阶段技能
      2-plan-workflows/ # 规划阶段技能
      3-solutioning/   # 方案阶段技能
      4-implementation/ # 实施阶段技能
    core-skills/      # 核心技能
  tools/
    installer/        # 安装工具
    web-bundles/      # Web集成包
  README_CN.md        # 中文文档
```

3.3 步骤 3：贪吃蛇网页游戏开发

3.3.1 项目结构

```
snake-game/
  index.html      # 主HTML文件
  style.css       # 样式文件
  script.js       # 游戏逻辑
  package.json    # 项目配置
  vercel.json     # Vercel部署配置
```

3.3.2 难度等级设计

难度等级	游戏速度 (ms)	描述
简单	150	适合新手，游戏节奏较慢
中等	100	默认难度，适合大多数玩家
困难	50	高难度，考验反应能力

表 4: 难度等级配置

3.3.3 界面主题选择

游戏支持多种界面主题风格：

主题名称	配色方案
经典紫	紫色渐变背景，绿色蛇身
海洋蓝	蓝色渐变背景，青色蛇身
森林绿	绿色渐变背景，棕色蛇身
日落橙	橙色渐变背景，红色蛇身

表 5: 界面主题配置

3.3.4 蛇的风格配置

风格名称	特点
方块蛇	经典方块造型，简单明了
渐变蛇	渐变色蛇身，视觉效果好
发光蛇	发光效果，现代感强
像素蛇	像素风格，怀旧复古

表 6: 蛇的风格配置

3.3.5 游戏核心代码

```
1 const canvas = document.getElementById('gameCanvas');
2 const ctx = canvas.getContext('2d');
3 const gridSize = 20;
4 const tileCount = canvas.width / gridSize;
5
6 let snake = [];
```

```
7 let food = {};  
8 let direction = { x: 0, y: 0 };  
9 let score = 0;  
10 let gameSpeed = 100;  
11 let gameLoop = null;  
12  
13 function initGame() {  
14     snake = [{ x: 10, y: 10 }, { x: 9, y: 10 }, { x: 8, y: 10 }];  
15     generateFood();  
16     direction = { x: 1, y: 0 };  
17     score = 0;  
18 }  
19  
20 function generateFood() {  
21     food = {  
22         x: Math.floor(Math.random() * tileCount),  
23         y: Math.floor(Math.random() * tileCount)  
24     };  
25 }  
26  
27 function update() {  
28     const newHead = {  
29         x: snake[0].x + direction.x,  
30         y: snake[0].y + direction.y  
31     };  
32  
33     if (checkCollision(newHead)) {  
34         gameOver();  
35         return;  
36     }  
37  
38     snake.unshift(newHead);  
39  
40     if (newHead.x === food.x && newHead.y === food.y) {  
41         score += 10;  
42         generateFood();  
43     } else {  
44         snake.pop();  
45     }  
46 }
```

```
47     draw();
48 }
49
50 function draw() {
51     ctx.fillStyle = '#f0f0f0';
52     ctx.fillRect(0, 0, canvas.width, canvas.height);
53
54     ctx.fillStyle = '#e74c3c';
55     ctx.fillRect(food.x * gridSize, food.y * gridSize, gridSize - 2,
56         gridSize - 2);
57
58     snake.forEach((segment, index) => {
59         if (index === 0) {
60             ctx.fillStyle = '#27ae60';
61         } else {
62             const gradient = index / snake.length;
63             ctx.fillStyle = `rgb(${100 + gradient * 100}, ${150 +
64                 gradient * 100}, 100)`;
65         }
66         ctx.fillRect(segment.x * gridSize, segment.y * gridSize,
67             gridSize - 2, gridSize - 2);
68     });
69 }
```

3.4 步骤 4：贪吃蛇游戏市场前景分析

3.4.1 市场规模与增长趋势

1. 休闲游戏市场：全球休闲游戏市场规模持续增长，预计 2026 年超过 500 亿美元
2. 网页游戏优势：无需下载安装，即时即玩，用户门槛低
3. 经典 IP 价值：贪吃蛇作为经典游戏 IP，具有广泛的用户基础和品牌认知度
4. 移动端适配：移动端游戏用户占比超过 70%，贪吃蛇适合移动端操作

3.4.2 目标用户群体

用户群体	特征
休闲玩家	年龄 18-35 岁，碎片化时间游戏需求
办公人群	工作间隙放松，解压需求
学生群体	课后娱乐，社交互动需求
中老年用户	简单易上手，怀旧情怀

表 7: 目标用户群体分析

3.4.3 盈利模式分析

盈利模式	说明
广告收入	游戏内展示广告，按曝光/点击计费
付费去广告	用户付费移除广告，提升体验
皮肤/主题付费	出售游戏皮肤、主题、蛇风格等虚拟物品
难度解锁	付费解锁高级难度或特殊模式
排行榜服务	付费查看全球排行榜或创建私人房间
品牌合作	与品牌联名推出限定主题

表 8: 盈利模式分析

3.4.4 竞争优势与差异化

- 1. 轻量化：单文件 HTML，加载速度快，无需安装
- 2. 可定制：支持多种主题和风格，满足个性化需求
- 3. 社交属性：支持多人联机和排行榜功能
- 4. 教育价值：可作为编程学习案例，吸引开发者社区
- 5. 跨平台：支持 PC、手机、平板等多种设备

3.4.5 市场化可行性评估

评估维度	评分 (1-5)	说明
市场需求	5	休闲游戏需求旺盛，贪吃蛇 IP 知名度高
技术可行性	5	技术成熟，开发难度低
成本投入	4	开发成本低，运营成本可控
盈利潜力	4	多种盈利模式，收入来源多样化
竞争程度	3	市场竞争激烈，需差异化定位
综合评分	4.2	具备良好的市场化潜力

表 9: 市场化可行性评估

3.5 步骤 5: Vercel 生产环境部署

3.5.1 网络连通性测试

在部署前，需测试 Vercel 服务的网络连通性：

```
1 curl -sS --max-time 12 -w "HTTP=%{http_code} | time=%{time_total}s\n"
   -o NUL "https://vercel.com"
2 curl -sS --max-time 12 -w "HTTP=%{http_code} | time=%{time_total}s\n"
   -o NUL "https://api.vercel.com"
3 curl -sS --max-time 12 -w "HTTP=%{http_code} | time=%{time_total}s\n"
   -o NUL "https://registry.npmjs.org"
```

3.5.2 安装 Vercel CLI

```
1 npm install -g vercel
2 vercel login
```

3.5.3 部署流程

```
1 cd frontend
2 npm install
3 npm run build
4 vercel --prod --yes
```

3.5.4 部署配置文件

创建 `vercel.json`:

```
1 {  
2   "version": 2,  
3   "builds": [  
4     {  
5       "src": "*.html",  
6       "use": "@vercel/static"  
7     }  
8   ],  
9   "routes": [  
10    {  
11      "src": "/*",  
12      "dest": "/*"  
13    }  
14  ]  
15 }
```

3.5.5 部署验证

部署成功后, Vercel 会返回部署 URL, 可通过以下命令验证:

```
1 curl -sS -w "HTTP={http_code}\n" -o NUL "https://your-project.vercel  
  .app"
```

3.6 步骤 6: MiKTeX 安装与 LaTeX 编译

3.6.1 MiKTeX 安装

1. 访问 <https://miktex.org/download> 下载 Basic MiKTeX Installer
2. 运行安装程序, 选择” 仅安装基本程序”
3. 在安装过程中选择” 自动安装缺失的包”
4. 完成安装后, 在命令行验证: `pdflatex --version`

3.6.2 Trae IDE LaTeX 插件安装

在 Trae IDE 中安装 LaTeX Workshop 插件, 配置如下:

```
1 {
2   "latex-workshop.latex.tools": [
3     {
4       "name": "xelatex",
5       "command": "xelatex",
6       "args": [
7         "-synctex=1",
8         "-interaction=nonstopmode",
9         "-file-line-error",
10        "%DOC%"
11      ]
12    }
13  ],
14  "latex-workshop.latex.recipes": [
15    {
16      "name": "xelatex",
17      "tools": ["xelatex"]
18    }
19  ]
20 }
```

3.6.3 PDF 编译命令

```
1 cd c:\Users\25590\Documents\123\pdf 编译器
2 xelatex experiment3.tex
3 xelatex experiment3.tex
```

3.7 步骤 7：银行 CRM 系统生成

3.7.1 系统架构设计

采用分层架构设计：

表示层 (Presentation Layer)

HTML5 / CSS3 / JavaScript / Tailwind CSS

业务逻辑层 (Business Layer)

Express.js / Node.js

数据访问层 (Data Layer)
MongoDB / MySQL

3.7.2 目录结构

银行系统/
 银行crm系统/
 server/
 models/
 Customer.js # 客户模型
 Followup.js # 跟进记录模型
 routes/
 customers.js # 客户管理路由
 followups.js # 跟进记录路由
 app.js # Express应用入口
 index.html # 前端主页面
 style.css # 样式文件
 script.js # 前端逻辑
 architecture.md # 架构设计文档
 prd.md # 产品需求文档
 package.json # 项目配置

3.7.3 核心功能模块

模块	功能描述
客户管理	客户信息 CRUD、搜索、筛选、导出
业务跟进	跟进记录管理、下次跟进提醒
数据分析	客户统计、业绩报表、可视化图表
认证授权	用户登录、角色权限管理
响应式 UI	支持 PC 和移动端访问

表 10: 银行 CRM 系统功能模块

3.7.4 数据库设计

客户表 (customers):

字段名	类型	索引	说明
no	String	UNIQUE	客户编号
name	String	-	客户姓名
gender	String	-	性别
phone	String	INDEX	手机号码
level	String	INDEX	资产等级
branch	String	INDEX	开户支行
manager	String	-	客户经理
createdAt	Date	-	创建时间

表 11: 客户表结构

跟进记录表 (followups):

字段名	类型	索引	说明
customerId	ObjectId	INDEX	关联客户 ID
type	String	-	跟进类型
content	String	-	跟进内容
nextFollowup	Date	-	下次跟进时间
createdAt	Date	-	创建时间

表 12: 跟进记录表结构

3.7.5 API 接口设计

API 路径	HTTP 方法	功能描述
/api/customers	GET	获取客户列表
/api/customers/:id	GET	获取单个客户
/api/customers	POST	创建客户
/api/customers/:id	PUT	更新客户
/api/customers/:id	DELETE	删除客户
/api/followups	GET	获取跟进记录
/api/followups	POST	创建跟进记录

表 13: API 接口设计

3.7.6 前端界面特点

- 1. 数据仪表盘：展示客户统计、业务数据等关键指标

- 2. 客户等级徽章：普通/金卡/白金卡/钻石卡不同颜色标识
- 3. 响应式布局：适配不同屏幕尺寸
- 4. 模态框交互：添加/编辑客户信息的弹窗操作
- 5. 搜索筛选：支持按姓名、电话、等级等条件筛选

4 实验结果

4.1 OpenBB Finance 技能测试结果

测试项	结果
技能安装	成功
股票数据获取	成功
技术指标计算	成功
AI 集成模式	成功
SpaceX 分析	成功

表 14: OpenBB 技能测试结果

4.2 BMAD 技能包安装结果

测试项	结果
项目克隆	成功
依赖安装	成功
技能包注册	成功
分析模块	可用
规划模块	可用
实施模块	可用

表 15: BMAD 技能包安装结果

4.3 贪吃蛇游戏功能清单

功能模块	实现状态
游戏核心逻辑	已完成
难度等级选择	已完成
界面主题切换	已完成
蛇的风格配置	已完成
键盘控制 (WASD/方向键)	已完成
分数系统	已完成
最高分记录	已完成
暂停/继续	已完成
响应式设计	已完成

表 16: 贪吃蛇游戏功能清单

4.4 银行 CRM 系统功能清单

功能模块	实现状态
客户管理 CRUD	已完成
跟进记录管理	已完成
数据仪表盘	已完成
客户搜索筛选	已完成
客户等级标识	已完成
响应式 UI	已完成
模态框交互	已完成
API 接口设计	已完成
数据库设计	已完成

表 17: 银行 CRM 系统功能清单

4.5 Vercel 部署结果

测试项	结果
网络连通性测试	通过
Vercel CLI 安装	成功
依赖安装	成功
项目构建	成功
生产环境部署	成功
访问验证	成功

表 18: Vercel 部署结果

4.6 项目文件统计

项目	文件数	说明
OpenBB 技能	6	SKILL.md, examples.md, scripts/
BMAD 技能包	50+	分析、规划、方案、实施模块
贪吃蛇游戏	5	HTML, CSS, JS, package.json, vercel.json
银行 CRM 系统	10	前端 + 后端 + 文档
合计	70+	-

表 19: 项目文件统计

5 问题与解决

1. **问题：** OpenBB 安装失败，提示 Python 版本不兼容 **解决：** 安装 Python 3.9-3.12 版本，确保版本符合要求
2. **问题：** BMAD 技能包安装后无法在 Trae IDE 中识别 **解决：** 检查技能包路径配置，确保正确注册到 IDE
3. **问题：** 贪吃蛇游戏难度切换不生效 **解决：** 在难度选择事件中重新设置游戏循环间隔
4. **问题：** Vercel 部署时构建失败 **解决：** 检查 package.json 中的 build 脚本，确保依赖安装完整
5. **问题：** MiKTeX 编译中文乱码 **解决：** 使用 xelatex 编译器，配置 ctexart 文档类
6. **问题：** 银行系统前端无法连接后端 API **解决：** 配置 CORS 跨域支持，确保前后端通信正常

6 实验心得

通过本次实验，我深入掌握了多项核心技术，取得了以下收获：

- 掌握了 OpenBB Finance 技能的使用，能够进行专业的金融数据分析和投资价值评估
- 理解了 BMAD 方法学的核心思想和技能包体系，学会了系统化的产品开发方法
- 开发了功能完善的贪吃蛇游戏，实现了难度等级、界面主题和蛇风格的多样化配置
- 对网页游戏的市场前景有了深入的认识，了解了盈利模式和竞争策略
- 掌握了 Vercel 生产环境部署流程，能够将 Web 应用发布到公网
- 学会了 MiKTeX 和 LaTeX 的安装配置，能够编译高质量的 PDF 文档
- 使用 BMAD 技能生成了完整的银行 CRM 系统，包括前端界面和后端 API

同时也认识到一些不足：

- SpaceX 作为私有公司，公开财务数据有限，分析深度受限
- BMAD 技能包的学习曲线较陡，需要更多实践
- 贪吃蛇游戏的社交功能需要进一步完善
- 银行系统缺少用户认证和权限管理模块

未来改进方向：深入研究 OpenBB 的高级功能，探索更多金融分析方法；完善贪吃蛇游戏的社交属性和多人联机功能；为银行系统添加完整的用户认证和数据安全机制；探索更多 BMAD 技能的应用场景。